

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING

SLC

Course Code:	CSA1402	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 - Imitation (L1)	Assessment	Assessment Tool 1 - ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	M. Madhu Rushwika	Reg. No.:	192371066
Section / Batch:	CSE	Date:	18-07-2026

Code of Conduct (192371066)
 I, M. Madhu Rushwika (Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
 Signature of the Student: M. Madhu Rushwika

- Instructions**
- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
 - Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
 - Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
 - Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 / 4	3 / 4	4 / 4	4 / 4	3 / 4	91.25
2	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
3	3 / 4	1 / 4	3 / 4	4 / 4	4 / 4	88.75
4	4 / 4	4 / 4	3 / 4	3 / 4	3 / 4	85
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						9
	/ 4	/ 4	/ 4	/ 4	/ 4	89
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N.D.H

b) Trace the processing input expression $p = (a * b) + (b * c) * 2$ through all phases of compiler

(a) Lexical Analysis

Lexeme

token

p

Identifier (ID)

=

Assignment operator

(

left parenthesis

a

Identifier (ID)

*

Multiplication operator

b

Identifier (ID)

)

Right parenthesis

+

Addition operator

(

left parenthesis

b

Identifier (ID)

*

Multiplication operator

(

Identifier (ID)

)

Right parenthesis

*

Multiplication operator

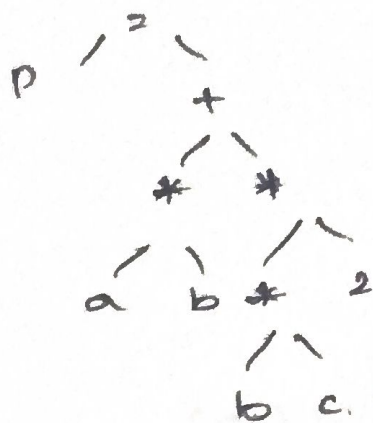
2

Numeric constant

eof

End of the file

B) Syntax Tree



c) Intermediate code

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$t_4 = t_1 + t_3$$

$$P = t_4$$

d) optimized intermediate code

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$P = t_1 + t_3$$

e) final target code

LOAD R1, a

MUL R1, b

LOAD R2, b

MUL R2, c

MUL R2, 2

ADD R1, R2

STORE P, R1

Significance of final target code:-

- 1) It is the machine language code executed by the processor
- 2) It performs arithmetic operations efficiently
- 3) It stores the final result variable P
- 4) That the last phase of compiler

2) For each of the following regular expression, construct the language tree valid string for each regular expression

a) $(a+aa)$

Language: A string formed using a (one, repeated on the notation)

1. a

2. aa

3. aaa

4. $aaaa$

5. $aaaaa$

b) $(a+ab)^*abb$

Language: A string over $\{a, b\}$ that end with abb

1. abb

2. $aabb$

3. $ababb$

4. $aaabbb$

5. $ababbb$

3) $(a+b^*)^* aba (a^*+b^*)^*$

language: all strings over $\{a,b\}$ that contain a substring

1. aba

2. abab

3. aabaa

4. baba

5. aababb

4) $(atab)^*$

language:- string formed by repeating

1. e

2. a

3. ab

4. aa

5. aab

5) $(aba)^* + (a^*b^*)^*$

language:- string that contains one or more occurrence of aba, followed by any combination of only a's (or) only b's

1. aba

2. abaa

3. abab

4. abaaba

5. abaabb

4) find regular expression corresponding each of the following subset $\{0, 1\}^*$:

i) The language of all strings have even number of 0's
even no. of 0's

Regular Expression - $(1^*(01^*01^*))^*$

ii) contain atleast one 1

Regular expression - $(0+1)^+1(0+1)^*$

iii) Both number of 0's & no. of 1's are odd

Regular expression - $(0+1)^*(01+10)(00+11)^*$

iv) Starts with 1 and end with 0

Regular expression - $1(0+1)^*0$

v) Does not consecutive of 1's

Regular expression - $(0+10)^*1?$

5) Consider lexical analyzer simplified programming language the language consists of the following the tokens.

Input string

int var123 = 12345;

float var456 = 123.45;

if (var123 == var456) {

3) find the no. of tokens following C statement is

i) No. of tokens

```
main ( )  
{  
    int a = 10;  
    int b = "abc";  
    int c = 30;  
    char d = "xyz";  
    float m = 40.5;  
}
```

comments are ignored

total tokens = 22

ii) Lexical Error

```
void main()  
{  
    int x = 10, y = 20;  
    char *a;  
    a = &x;  
    x = 1xab;  
}
```

1. 1xab → Invalid identifier

2. a = &x; → No lexical error

Answer: Lexical error is 1xab

a) Total no. of tokens = 20

S: No

S: No	Token	Type
1	int	keyword
2	val123	identifiers
3	=	operator
4	12345	integer literal
5	;	separator
6	float	keyword
7	val456	identifiers
8	=	operator
9	123.45	floating-point literal
10	;	separator
11	if	keyword
13	val123	separator
14	=	operator
15	val456	identifiers
16	;	separator
17	{	separator
18	return	keyword
19	;	separator
20	}	separator

B) categorized tokens

keyword: int, float, if, return

identifiers: ;, ,, (), {, }, }

integer literal: 12345

float point literal: 123.45

C) FSM table transitions:

Approximate transitions

keywords $\rightarrow 18$

identifiers $\rightarrow 24$

integer literal $\rightarrow 5$

float point literal $\rightarrow 6$

operator $\rightarrow 6$

separator $\rightarrow 4$

Estimated total FSM state transition = 64

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate, correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited unclear interpretation or	No interpretation